Thread exercises

State premises and intermediate states before reading the separate solutions.

OS04-E01 Sharing

Two threads in one process each execute counter++ once on a global integer initially 0. Both operations complete without integer overflow.

1. Why can the abstract increments produce only one increase?
2. What does TLS remove, and what does it lose?
3. How does a mutex change observable behavior?

OS04-E02 Concurrency and speedup

For the same fixed problem, 80% of baseline time is ideally parallel and 20% serial. Use eight cores without overhead.

1. Compute the Amdahl speedup bound.
2. Why can measurement be lower?
3. Recompute with 5% serial time.

OS04-E03 Cancellation and cleanup

A worker owns mutex M and buffer B and waits in pthread_cond_wait on a stable predicate while awaiting a network reply. With cancellation disabled, it has already registered cleanup handlers. Main requests cancellation.

1. How can asynchronous cancellation stall the system?
2. How does deferred cancellation support cleanup?
3. What additional design is required?

OS04-E04 M:N compensation

One hundred user threads share ten fixed carriers, without asynchronous I/O or replacements. Each wants a blocking syscall.

1. How many carriers remain after one or ten block?
2. Why is ten blocked user threads not a universal M:N maximum?
3. Compare many-to-one and one-to-one.

OS04-E05 Creation, join, and TLS

Main owns Work{input14,output}; worker writes output=3×input. A two-worker pool also repeatedly handles different jobs.

1. When may main read 42 without conflict? Is sleeping 10 ms enough?
2. Distinguish successful wait and error branches in Pthreads, Windows, and Java.
3. Assess returning a worker stack pointer or freeing parameters immediately after detach.
4. How do TLS counters and per-job state differ?

OS04-E06 Amdahl, DAG, and false sharing

Same problem, T1=100 ms=20 serial+80 parallel. A perfect four-level binary sum tree pairs adjacent leaves 1…8, with unit leaf and merge cost. Lines are 64 B, integers 4 B, base aligned.

1. Give ideal P8 time, speedup, efficiency, and infinite-core limit.
2. Why can 80% of operations differ from 80% of time?
3. Give intermediate sums, work/span, and general lower bound.
4. Does W1 writing a[0] and W2 writing a[1] constitute a data race?

OS04-E07 Pool and serial-queue waits

Two workers run A/B. Each submits A1/B1 to the same pool and blocks waiting. There is no help-join.

1. Why can an acyclic task DAG still stall?
2. Give fixes without assuming universal work stealing.
3. What if a block on serial Q synchronously submits to Q and waits?
4. Assume exactly four workers process [1,2], [3,4], [5,6], and [7,8]. Give their partial sums and one valid reduction order.

OS04-E08 Cancellation, fork, and signals

Worker owns M and B, pushes unlock-M then free-B handlers, and waits on a stable predicate in pthread_cond_wait. In another process, threads T1 and T2 belong to that same process; T1 forks while T2 owns a lock.

1. Does successful cancellation request prove termination?
2. Give complete cond-wait cancellation resource order.
3. Can unlocking alone always repair partial updates?
4. Describe the fork child's threads and signal mask/disposition.

OS04-E09 Historical object counts

Windows 7 CreateProcessW succeeds, followed by one CreateThread. Linux 3.2 starts with one ordinary user task and no others; compare a successful ordinary fork against a successful same-group clone with VM/FS/FILES/SIGHAND/THREAD.

1. How many Windows threads and TEBs exist?
2. Count Linux task/mm/page tables/files in each mode.
3. Does VM/FS/FILES/SIGHAND with SIGCHLD imply one thread group?
4. Are writes to a shared-mm heap automatically safe?

OS04-E10 Scheduling entities and CPU shares

On one CPU, P has three continuously runnable threads and Q has two. All kernel scheduling entities have equal priority and receive equal long-run CPU shares. In many-to-one, each process has one carrier; in one-to-one, every thread has its own kernel entity.

1. Find the CPU share of P and Q under each mapping.
2. Why does this not prove that adding threads always increases a process's CPU share? Give a policy or workload change that alters the result.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 4. Linux 3.2 fork.c; pthread_join(3); pthread_cond_wait(3p); C++ thread storage; OpenMP reduction.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study